Reviewer Pack — Minimal Root System Length of Coxeter Groups vs Circuit Comp…

Entry 5dae2dfdf4ab · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 04:34:16 UTC

Minimal Root System Length of Coxeter Groups vs Circuit Complexity for AND-OR Trees

Entry ID: `5dae2dfdf4ab` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 04:34:16 UTC

1. Conjecture

Field A (mathematical branch): Coxeter Group Theory **Field B** (complexity object): Complexity Theory: Circuit Complexity for AND-OR Trees

Statement:

> ["For any AND-OR tree T with n variables, the minimal root system length of the Coxeter group associated with the automorphism group of the tree's structure is Θ(log n).", 'Further, this length provides an upper bound on the depth of T.', 'Conversely, for any Coxeter group G with minimal root system length ≤ log n, there exists an AND-OR tree T with automorphism group G and depth ≤ 2^n.']

Rationale (proposer's reasoning):

> ['Coxeter groups have been used to model symmetry in various mathematical structures, including geometric configurations and graph structures. Their algebraic properties may provide insights into the complexity of computational problems.', 'AND-OR trees are a fundamental structure for reasoning about logical formulas. A connection between Coxeter groups and AND-OR trees could reveal new relationships between symmetry and circuit complexity.', 'This conjecture suggests that the minimal root system length, which captures the symmetry information of the automorphism group, is related to the computational complexity of evaluating the AND-OR tree.']

Taxonomy category: `coxAx` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1cf6ff74946dd21b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if at least 90% of randomly generated AND-OR trees T with n variables have Coxeter groups with minimal root system length within a factor of 1.5 from Θ(log n), and their depths are ≤ log n. The conjecture is falsified if any tree has a

depth greater than log n or the minimal root system length is more than 50% higher than Θ(log n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random

def generate_and_or_tree(n):
    if n == 0:
        return 'L'
    elif n == 1:
        return 'O'
    else:
        left_size = random.randint(0, n-2)
        right_size = n - 1 - left_size
        return ('O', generate_and_or_tree(left_size), generate_and_or_tree(right_size))

def compute_coxeter_number(tree):
    if tree == 'L':
        return 1
    elif tree == 'O':
        return 2
    else:
        left_num = compute_coxeter_number(tree[1])
        right_num = compute_coxeter_number(tree[2])
        return max(left_num, right_num) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""
```

```
    for n in n_values:
        for _ in range(5):
            tree = generate_and_or_tree(n)
            coxeter_num = compute_coxeter_number(tree)
            expected_log_n = math.log2(n + 1)
            if coxeter_num < expected_log_n * 0.75 or coxeter_num > expected_log_n * 1.25:
                conjecture_holds = False
                counterexample = f"n={n}, tree={tree}, coxeter_num={coxeter_num}"
                break
            total_metric_value += abs(coxeter_num - expected_log_n)
            instances_tested += 1

    return {
        "metric_name": "Coxeter Number vs Log(n)",
        "metric_value": total_metric_value / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(res
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    elif support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61117344.py", line 72, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_61117344.py", line 60, in run_trial
    "metric_value": total_metric_value / instances_tested,
```

```
                    ~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing any data, which prevents us from verifying the conjecture's support conditions. | next: Re-run the test to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13056
2	propose	ollama_remote	glm4:latest	0	0	10591
3	preregistration	ollama_remote	glm4:latest	0	0	6197
4	novelty	ollama_remote	glm4:latest	0	0	4764
5	novelty	ollama_remote	glm4:latest	0	0	5412
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26140
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10044
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9513
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8718
10	judge	ollama_remote	glm4:latest	0	0	8145

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 102580 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5dae2dfdf4ab.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5dae2dfdf4ab.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5dae2dfdf4ab.tar.gz` (if generated)